Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

Proyecto 2 - Sistema de Control de Drones

202400554 – Raúl José Daniel Paredes González

Resumen

El presente proyecto consiste en el desarrollo de un sistema de control de drones para la transmisión de mensajes encriptados mediante luces LED. El sistema permite que múltiples drones se desplacen verticalmente y emitan luz a diferentes alturas, donde cada combinación de dron y altura representa una letra del alfabeto según una tabla de codificación configurable.La solución implementa una arquitectura MVC (Modelo-Vista-Controlador) desarrollada en C# con ASP.NET Core. Se diseñaron e implementaron Tipos de Datos Abstractos (TDAs) propios, incluyendo listas simplemente enlazadas (ListaDrones,ListaMensajes,ListaSistemasDrones) , una cola FIFO (ColaInstrucciones) y una tabla de codificación (TablaCodificacion), cumpliendo estrictamente con la restricción de no utilizar estructuras de datos nativas de C# como List<T> o Queue<T>.

Palabras clave

1. Tipos de Datos Abstractos
2. Drones
3. Graphviz
4. Listas enlazadas
5. POO

Abstract

his project involves the development of a drone control system for transmitting encrypted messages using LED lights. The system allows multiple drones to move vertically and emit light at different heights, where each combination of drone and height represents a letter of the alphabet according to a configurable encoding table. The solution implements an MVC (Model-View-Controller) architecture developed in C# with ASP.NET Core. Custom Abstract Data Types (ADTs) were designed and implemented, including singly linked lists (DroneList, MessageList, DroneSystemList), a FIFO queue (InstructionQueue), and an encoding table (EncodingTable), strictly complying with the restriction of not using native C# data structures such as List<T> or Queue<T>.

Keywords

1. Abstract Data Types

2. Drones

3. Graphviz

4. Linked Lists

5. Object-Oriented Programming

Introducción

En el ámbito de la seguridad y comunicaciones, la transmisión de información sensible requiere mecanismos que garanticen su integridad y confidencialidad. El presente proyecto aborda esta problemática mediante la implementación de un sistema que utiliza un conjunto de drones para transmitir mensajes encriptados a través de luces LED, donde cada combinación de dron y altura representa una letra del alfabeto.

El sistema desarrollado permite cargar configuraciones desde archivos XML, gestionar drones, sistemas de codificación y mensajes, así como calcular el tiempo óptimo de transmisión mediante un planificador que coordina el movimiento paralelo de múltiples drones. La solución implementa conceptos fundamentales de programación orientada a objetos (POO) y Tipos de Datos Abstractos (TDAs) propios, cumpliendo con las restricciones establecidas..

1. Programación Orientada a Objetos en C#

La Programación Orientada a Objetos (POO) constituyó la base arquitectónica del sistema. Este paradigma permite organizar el código en torno a entidades del mundo real, facilitando su mantenimiento y escalabilidad. En palabras de Joyanes (2020), "la POO proporciona mecanismos de abstracción que permiten modelar problemas complejos mediante la identificación de clases y objetos".

Tabla I.

Clases Implementadas

Clase	Descripción
Dron	Representa un dron con nombre, altura actual y acciones (Subir, Bajar, Esperar, Emitir Luz)
SistemaDrones	Agrupa un conjunto de drones y su tabla de codificación (dron + altura = letra)
Mensaje	Contiene un conjunto de instrucciones que forman un texto a transmitir
InstruccionEmision	Relaciona un dron con una altura objetivo para representar una letra
PlanVuelo	Almacena la secuencia optimizada de acciones por segundo

Fuente: elaboración propia.

2. Tipos de Datos Abstractos (TDAs)

Un Tipo de Dato Abstracto es un modelo que define valores y las operaciones que se pueden realizar sobre ellos, ocultando los detalles de implementación. (SoyHenry, 2022)

Tabla 2.

TDAs Implementados.

TDA	Descripción
ListaDrones	Lista simplemente enlazada para almacenar drones
ListaMensajes	Lista enlazada para almacenar mensajes
ListaSistemasDrones	Lista enlazada para almacenar sistemas
ColaInstrucciones	Cola FIFO para procesar instrucciones en orden
TablaCodificacion	Estructura para mapear (dron, altura) -> letra

Fuente: elaboración propia

3. Procesamiento de Archivos XML

Los archivos XML constituyeron el medio de entrada y salida de información. El lenguaje de marcado extensible (XML) permite estructurar datos de manera jerárquica, facilitando su procesamiento automatizado (Pérez, 2020).

El archivo de entrada contiene la estructura siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<pacientes>
   <paciente>
      <datospersonales>
         <nombre>Lucas</nombre>
         <edad>33</edad>
      </datospersonales>
      <periodos>15</periodos>
      <m>10</m>
      <rejilla>
         <celda f="1" c="1" />
         <celda f="2" c="1" />
         <celda f="3" c="1" />
         <celda f="4" c="1" />
         <celda f="5" c="1" />
         <celda f="6" c="1" />
         <celda f="7" c="1" />
         <celda f="8" c="1" />
         <celda f="9" c="1" />
         <celda f="10" c="1" />
         <celda f="1" c="10" />
         <celda f="2" c="10" />
         <celda f="3" c="10" />
         <celda f="4" c="10" />
         <celda f="5" c="10" />
         <celda f="6" c="10" />
         <celda f="7" c="10" />
         <celda f="8" c="10" />
         <celda f="9" c="10" />
         <celda f="10" c="10" />
      </rejilla>
   </paciente>

</pacientes>
```

Figura 1. Entrada XML.

Fuente: Universidad San Carlos de Guatemala Facultad de Ingeniería Escuela de Ciencias y Sistema.

El sistema parsea este documento utilizando las clases XmlDocument y XmlNode de .NET, extrayendo la información para construir objetos Paciente y sus rejillas asociadas. Las celdas contagiadas se almacenan en una ListaCelda, asumiendo que las no mencionadas están sanas.

4. Visualización con Graphviz

Graphviz es una herramienta de código abierto para la representación gráfica de estructuras de datos mediante el lenguaje DOT (Takeyas, 2012). En este proyecto, se utilizó para generar imágenes de las rejillas en cada período de simulación.

El proceso consiste en:

1. Crear un archivo .dot con una tabla HTML que representa la rejilla.
2. Las celdas contagiadas se colorean en rojo.
3. Ejecutar el comando dot -Tpng para convertir el archivo a formato PNG.

Esta representación visual permite a los científicos observar directamente cómo evoluciona el patrón de contagio, facilitando la identificación de comportamientos periódicos. La Figura 1 muestra un ejemplo de rejilla generada.

5. Algoritmo de Planificación

El planificador implementa un algoritmo que optimiza el tiempo de transmisión permitiendo que múltiples drones se muevan en paralelo, respetando que solo un dron puede emitir luz por segundo

Conclusiones

1. Integración exitosa de POO y TDAs: El proyecto demuestra cómo la combinación de programación orientada a objetos y Tipos de Datos Abstractos propios permite construir una solución modular, extensible y mantenible.

2. Optimización del tiempo de transmisión: El algoritmo de planificación implementado logra reducir el tiempo total de transmisión al permitir el movimiento paralelo de múltiples drones, respetando la restricción de una emisión por segundo.

3. Flexibilidad mediante XML: El uso de archivos XML para la configuración inicial y la generación de reportes brinda flexibilidad al sistema, permitiendo cargar diferentes configuraciones sin modificar el código fuente.

4. Visualización de datos: La integración con Graphviz proporciona una capa visual que facilita la comprensión de las relaciones entre drones y la secuencia de instrucciones optimizada.

Referencias bibliográficas

Cairó, O. (2018). *Estructuras de datos*. McGraw Hill Interamericana.

Joyanes,L. (2020). *Programación orientada a objetos con C#*. McGraw-Hill.

Pérez, C. (18 de diciembre de 2020). Qué es y cómo abrir un archivo XML. *MuyInteresante*. https://www.muyinteresante.es/tecnologia/23571.html

Takeyas,B. L. (2012). *Estructura de datos orientados a objetos: Pseudocódigo y aplicaciones en C#.NET*. Alfa Omega.

Anexos

1. Diagrama de Clases

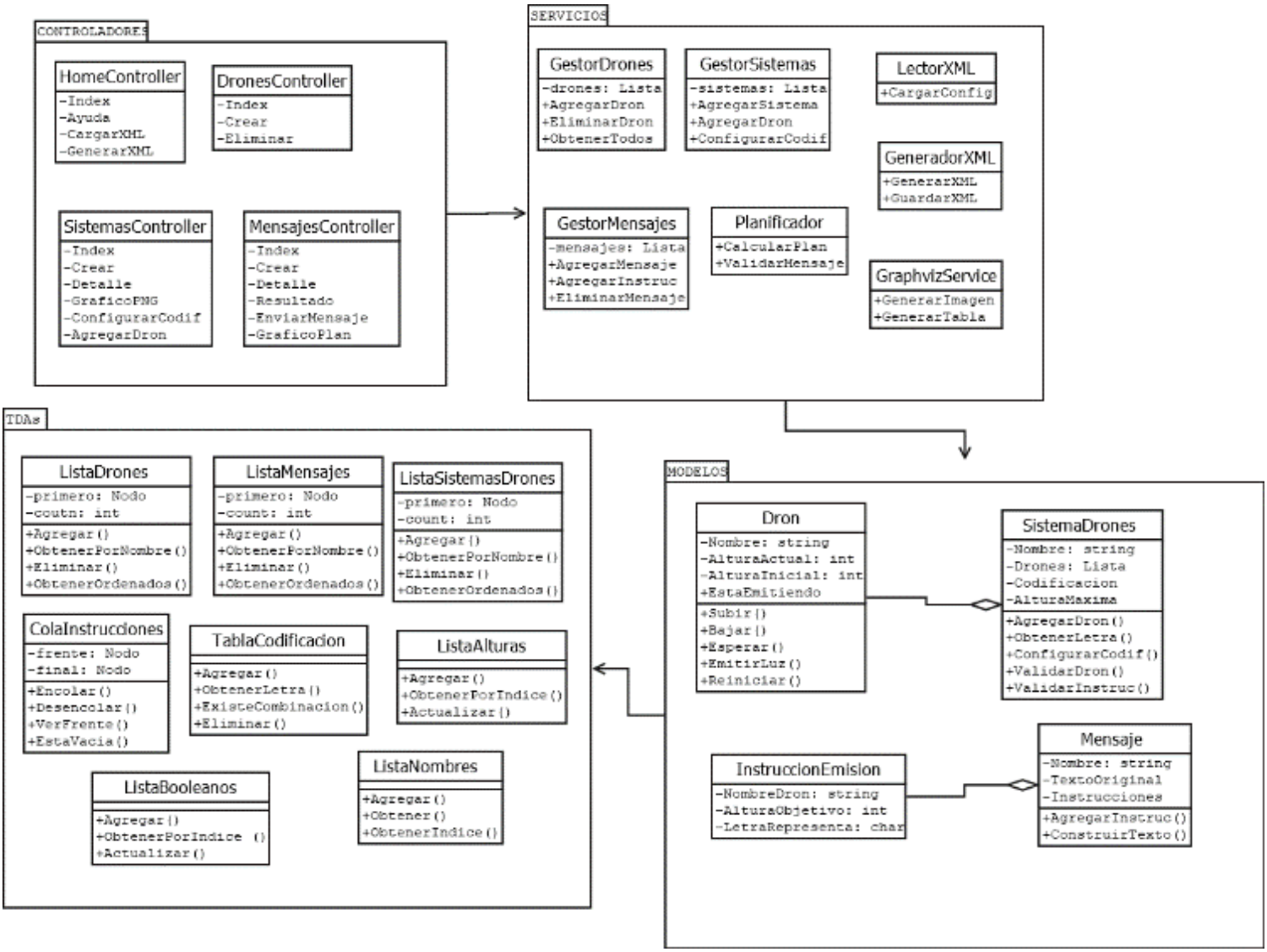

Figura 1. Diagrama de clases.

Fuente: Imagen propia.

2. Diagrama de Flujo

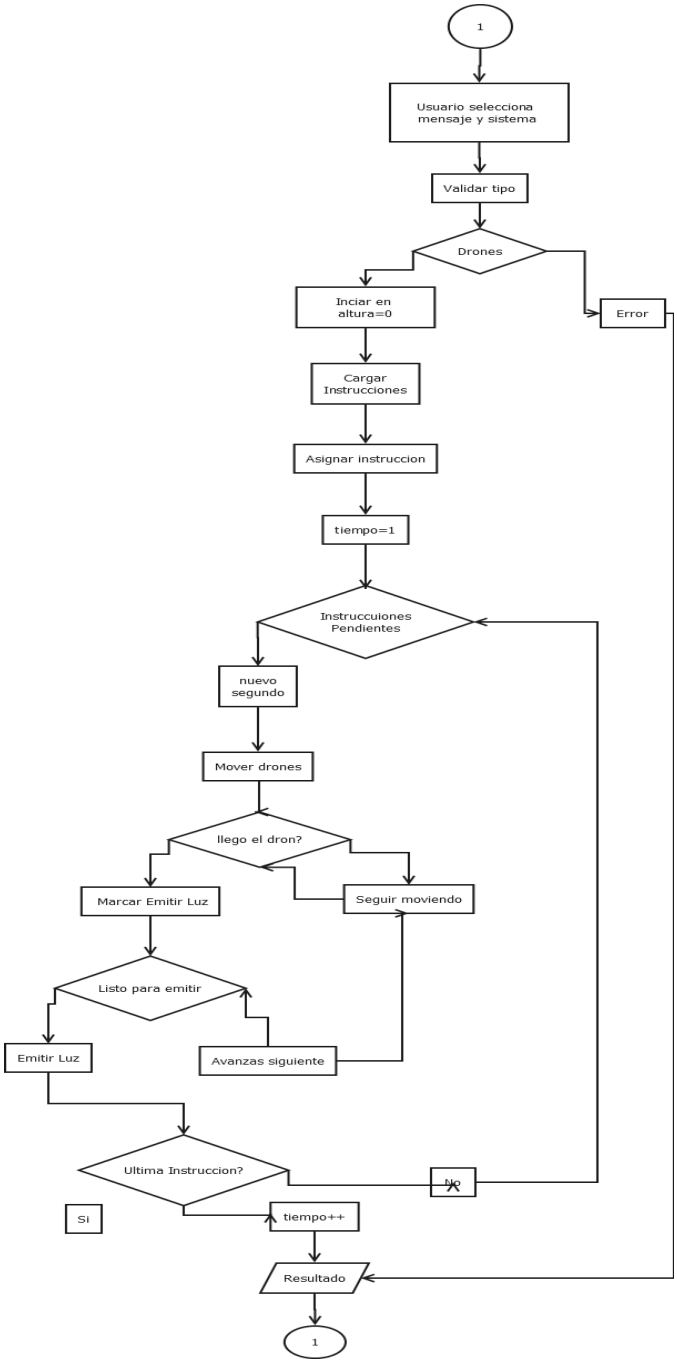

Figura 2. Diagrama de Flujo.

Fuente: Imagen propia.

3. Diagrama de Actividades

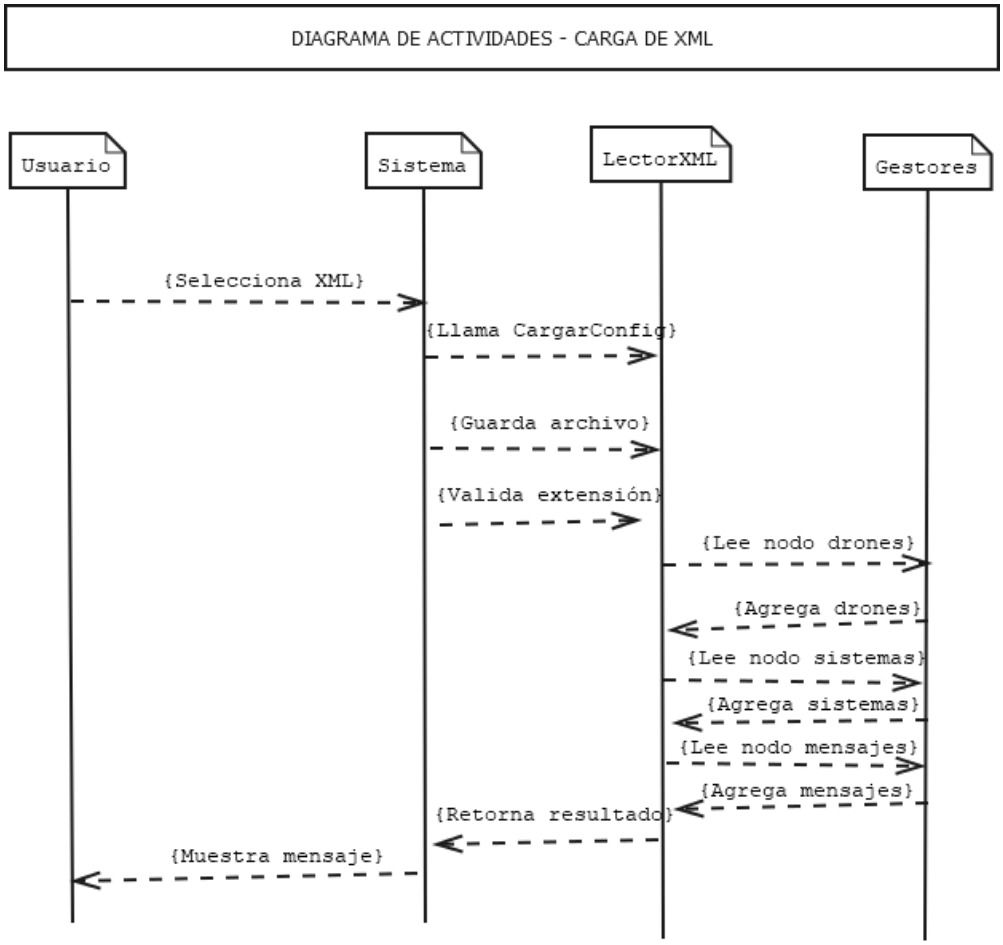

Figura 3. Diagrama Actividades

Fuente: Imagen propia.